claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-adf29828549d3025f.jsonl`
- SHA-256: `8dd58064ee7bba20ae92a95ef4229a254b6e43e2c602570ccf3da56bf6433f53`
- Source modified: `2026-07-07T04:12:16+00:00`
- Imported at: `2026-07-08T16:00:26+00:00`
- project: `wf_ddb0b3fd-b40`
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`

Transcript

1. user (2026-07-07T04:10:28.572Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:
- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts): TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:
- file: backend/pipeline/detectors/native_wasb_runner.py
- line: 327
- summary: run_predict still appends --prefetch-batches when decode_backend=nvdec_resident, but wasb_infer.py's nvdec branch (main() lines 1308-1319) never threads args.prefetch_batches into run_video_nvdec_resident, so the configured knob is silently inert.
- failure scenario: On the documented rollout path ? cloud-serving preset (backend/config/presets.py:60 sets wasb.prefetch_batches=4) plus WASB_DECODE_BACKEND=nvdec_resident ? the subprocess argv carries '--prefetch-batches 4', argparse accepts it, and the value is dropped on the floor with no log; an operator tuning prefetch_batches to change throughput sees no effect and no warning, unlike every other knob the runner deliberately passes 'only when meaningful' (comment at lines 322-323).
- severity: minor

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario actually happens in the code as written. Default to isReal=false if the scenario cannot occur,

```
is already guarded, or rests on a misreading.
```

2. user (2026-07-07T04:10:37.594Z)

```
1091 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch
```

3. user (2026-07-07T04:10:40.236Z)

```
85-    timeout_sec: int = 1800  # 30 min; a hung GPU call is killed (0 = no timeout)
86-    keep_frames: bool = (
87-        False  # retain the decoded frame cache after success (debug only)
88-    )
89-    # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long clip's
90-    # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a real
GPU
91-    # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
92-    stream_video: bool = True
93:    # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
94:    # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset opts
in.
95-    batch_size: int = 0
96:    prefetch_batches: int = 0
97-    # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend / WASB_DECODE_BACKEND).
98-    # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths, argv
99-    # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env only).
100-    decode_backend: str = "cpu"
101-
102-    @classmethod
103-    def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
104-        if isinstance(idx_cfg, dict):
--
117-            scratch_dir=env("WASB_SCRATCH")
118-            or env("RALLY_SCRATCH")
119-            or env("TMPDIR")
120-            or "",
121-            timeout_sec=w.timeout_sec,
122-            keep_frames=w.keep_frames,
123-            stream_video=(True if sv is None else bool(sv)),
124-            batch_size=int(w.batch_size or 0),
125:            prefetch_batches=int(w.prefetch_batches or 0),
126-            decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
127-        )
128-
129-
130-class NativeWasbRunner(DetectorRunner):
131-    """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
132-
133-    def __init__(self, cfg: NativeWasbConfig):
--
318-            device,
319-            "--out",
320-            out_csv,
321-        ]
322-        # GPU-feed tuning: pass only explicit non-zero values so the default argv (and its
323-        # byte-parity guarantee) is unchanged when the knobs are unset.
324-        if self.cfg.batch_size:
325-            argv += ["--batch-size", str(self.cfg.batch_size)]
326:        if self.cfg.prefetch_batches:
327:            argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
328-
329-        logger.info(
330-            "Running native WASB inference (device=%s%s) on %s ...",
```

```
331-            device,
332-            " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
333-            os.path.basename(norm),
334-        )
335-        try:
```

4. user (2026-07-07T04:10:47.692Z)

```
496-            except BaseException as e:  # noqa: BLE001 - surfaced in __next__
497-                self._exc = e
498-            finally:
499-                self._q.put(self._END)
500-
501:        self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
502-        self._t.start()
503-
504-    def __iter__(self):
505-        return self
506-
--
526-    flush_every: int = 1000,
527-    fresh: bool = False,
528-    frame_loader=None,
529-    source_key: "str | None" = None,
530-    device: str = "cuda",
531:    prefetch_batches: int = 0,
532-) -> str:
533-    """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
534-
535-    Two equivalent ways to supply pixels (output is bit-identical between them):
536-
--
697-                        )
698-                    )
699-                # Overlap CPU batch assembly with GPU inference (streaming/workers=0 only ?
700-                # a multi-worker DataLoader already overlaps via its worker processes).
701-                batches = (
702:                    _PrefetchIter(loader, depth=prefetch_batches)
703:                    if prefetch_batches and loader_workers == 0
704-                    else loader
705-                )
706-                for bi, batch in enumerate(batches):
707-                    imgs, _hms, trans = batch[0], batch[1], batch[2]
708-                    img_paths = [
--
877-    log_every_batches: int = 50,
878-    cache_dir: "str | None" = None,
879-    flush_every: int = 1000,
880-    fresh: bool = False,
881-    device: str = "cuda",
882:    prefetch_batches: int = 0,
883-) -> str:
884-    """Fast path: decode ``video_path`` on the fly and feed frames straight to the
885-    detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
886-    dominates a long clip). Output is bit-identical to the frames-on-disk path; see
887-    ``run``'s docstring for why.
--
908-            flush_every=flush_every,
909-            fresh=fresh,
910-            frame_loader=frame_loader,
911-            source_key=source_key,
912-            device=device,
913:            prefetch_batches=prefetch_batches,
914-        )
915-    finally:
```

```
916-        release()
917-
918-
--
1268-        "--limit", type=int, default=0, help="cap #frames (for quick tests)"
1269-    )
1270-    ap.add_argument("--batch-size", type=int, default=8)
1271-    ap.add_argument("--num-workers", type=int, default=4)
1272-    ap.add_argument(
1273:        "--prefetch-batches",
1274-        type=int,
1275-        default=0,
1276-        help="streaming mode: overlap CPU batch assembly with GPU inference via a "
1277-        "bounded producer thread of this queue depth (0 = off, the parity default)",
1278-    )
--
1332-            log_every_batches=args.log_every_batches,
1333-            cache_dir=args.cache_dir,
1334-            flush_every=args.flush_every,
1335-            fresh=args.fresh,
1336-            device=args.device,
1337:            prefetch_batches=args.prefetch_batches,
1338-        )
1339-        return
1340-
1341-    frames_dir = args.frames_dir
1342-    if args.video:
```

5. user (2026-07-07T04:10:48.502Z)

```
54-        # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid L4
55-        # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56-        # CPU-feed-bound. Serving opts into the measured fast values; local/WSL unchanged.
57-        "indexing": {
58-            "detector_impl": "native",
59-            "skip_ai_handoff": False,
60:            "wasb": {"batch_size": 64, "prefetch_batches": 4},
61-        },
62-        "storage": {"backend": "gcs"},
63-    },
64-    "cpu-mock": {
65-        "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
66-        # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
```

6. user (2026-07-07T04:10:56.772Z)

```
1240            "--frames_out_dir", help="where to extract frames when --video is used"
1241        )
1242        ap.add_argument(
1243            "--stream-video",
1244            action="store_true",
1245            help="FAST PATH: decode --video on the fly and feed frames straight to "
1246            "the detector (no PNG extraction). Output-equivalent to the disk path.",
1247        )
1248        ap.add_argument(
1249            "--decode-backend",
1250            default="cpu",
1251            choices=list(DECODE_BACKENDS),
1252            help="cpu (default) = the historical cv2 decode paths (disk frames or "
1253            "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
1254            "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "
1255            "~1.7x on an L4. Requires --video; detections cache separately from cpu "
1256            "(trajectory-level, not byte-level, equivalence).",
1257        )
1258        ap.add_argument("--weights", required=True)
```

```
        ap.add_argument("--out", required=True)
        ap.add_argument("--sport", default="badminton")
        ap.add_argument(
            "--device",
            default="cuda",
            choices=["cuda", "mps", "cpu"],
            help="torch device for the detector (default cuda = the WSL parity path)",
        )
        ap.add_argument(
            "--limit", type=int, default=0, help="cap #frames (for quick tests)"
        )
        ap.add_argument("--batch-size", type=int, default=8)
        ap.add_argument("--num-workers", type=int, default=4)
        ap.add_argument(
            "--prefetch-batches",
            type=int,
            default=0,
            help="streaming mode: overlap CPU batch assembly with GPU inference via a "
            "bounded producer thread of this queue depth (0 = off, the parity default)",
        )
        ap.add_argument(
            "--log-every-batches", type=int, default=50, help="progress log cadence"
        )
        ap.add_argument(
            "--cache-dir",
            default=None,
            help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
        )
        ap.add_argument(
            "--flush-every",
            type=int,
            default=1000,
            help="fsync the detector cache every N windows (bounds reboot loss)",
        )
        ap.add_argument(
            "--fresh", action="store_true", help="ignore any existing cache and start over"
        )
        args = ap.parse_args()

        if args.decode_backend == "nvdec_resident":
            # GPU-resident branch FIRST: it decodes the video itself (NVDEC), so neither the
            # --stream-video cv2 path nor the extract-frames fallback below may run.
            if not args.video:
                raise SystemExit("--decode-backend nvdec_resident requires --video")
            if args.device != "cuda":
                raise SystemExit(
                    "--decode-backend nvdec_resident requires --device cuda "
                    "(NVDEC decode runs on the GPU)"
                )
            run_video_nvdec_resident(
                args.video,
                args.weights,
                args.out,
                args.sport,
                args.limit,
                batch_size=args.batch_size,
                log_every_batches=args.log_every_batches,
                cache_dir=args.cache_dir,
                flush_every=args.flush_every,
                fresh=args.fresh,
            )
            return

        if args.stream_video:
            if not args.video:
```

```
1324                raise SystemExit("--stream-video requires --video")
1325            run_video_streaming(
1326                args.video,
1327                args.weights,
1328                args.out,
1329                args.sport,
1330                args.limit,
1331                batch_size=args.batch_size,
1332                log_every_batches=args.log_every_batches,
1333                cache_dir=args.cache_dir,
1334                flush_every=args.flush_every,
1335                fresh=args.fresh,
1336                device=args.device,
1337                prefetch_batches=args.prefetch_batches,
1338            )
1339            return
1340
1341        frames_dir = args.frames_dir
1342        if args.video:
1343            frames_dir = extract_frames(
1344                args.video, args.frames_out_dir or (args.video + "_frames")
1345            )
1346        if not frames_dir:
1347            raise SystemExit("provide --frames_dir or --video")
1348        run(
1349            frames_dir,
1350            args.weights,
1351            args.out,
1352            args.sport,
1353            args.limit,
1354            batch_size=args.batch_size,
1355            num_workers=args.num_workers,
1356            log_every_batches=args.log_every_batches,
1357            cache_dir=args.cache_dir,
1358            flush_every=args.flush_every,
1359            fresh=args.fresh,
1360            device=args.device,
1361        )
1362
1363
1364    if __name__ == "__main__":
1365        main()
1366
```

7. user (2026-07-07T04:11:11.977Z)

```
250             # instead of letting wasb_infer.py die mid-run on a paid box.
251             if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
252                 return False, (
253                     f"decode_backend=nvdec_resident requires CUDA, but the effective device
is "
254                     f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
255                 )
256             return True, out
257
258         def run_predict(
259             self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
260         ) -> Optional[str]:
261             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
262
263             Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
264             runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
```

```
265             same downstream contract, so the parity comparison is apples-to-apples.
266             """
267             output_dir = os.path.abspath(output_win_dir)
268             os.makedirs(output_dir, exist_ok=True)
269             # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
270             norm = str(video_win_path).replace("\\", "/")
271             stem, _ext = os.path.splitext(os.path.basename(norm))
272             # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
273             key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
274             out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
275
276             if not self._copy_infer_script():
277                 logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
278                 return None
279
280             weights = os.path.expanduser(self.cfg.weights_path)
281             argv = [
282                 self.cfg.python_bin,
283                 "wasb_infer.py",
284                 "--video",
285                 os.path.abspath(str(video_win_path)),
286             ]
287             frames_dir: Optional[str] = None
288             if self.cfg.decode_backend == "nvdec_resident":
289                 # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
itself
290                 # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
neither
291                 # --stream-video nor --frames_out_dir applies and there is no frame cache to
clean.
292                 argv += ["--decode-backend", self.cfg.decode_backend]
293             elif self.cfg.stream_video:
294                 # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
295                 argv.append("--stream-video")
296             else:
297                 scratch = self.cfg.scratch_dir or output_dir
298                 frames_dir = os.path.join(scratch, f"{key}_frames")
299                 argv += ["--frames_out_dir", frames_dir]
300             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
301             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
302             device = self._effective_device()
303             if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
304                 # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on
a
305                 # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
subprocess.
306                 logger.error(
307                     "decode_backend=nvdec_resident requires CUDA, but the effective device
is "
308                     "%r ? aborting before the WASB subprocess.",
309                     device,
310                 )
311                 return None
312             argv += [
313                 "--weights",
314                 weights,
315                 "--sport",
316                 self.cfg.sport,
317                 "--device",
318                 device,
319                 "--out",
```

```
320                 out_csv,
321             ]
322             # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
323             # byte-parity guarantee) is unchanged when the knobs are unset.
324             if self.cfg.batch_size:
325                 argv += ["--batch-size", str(self.cfg.batch_size)]
326             if self.cfg.prefetch_batches:
327                 argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
328
329             logger.info(
330                 "Running native WASB inference (device=%s%s) on %s ...",
331                 device,
332                 " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
333                 os.path.basename(norm),
334             )
335             try:
336                 res = self._run(argv, cwd=self._repo_src())
337             except FileNotFoundError:
338                 logger.error(
339                     "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
340                 )
341                 return None
342             except subprocess.TimeoutExpired:
343                 logger.error(
344                     "WASB native inference timed out after %ss.", self.cfg.timeout_sec
345                 )
346                 return None
347             if res.returncode != 0:
348                 logger.error(
349                     "WASB native inference failed:\n%s",
350                     (res.stderr or res.stdout or "")[-2000:],
351                 )
352                 return None
353             if not os.path.exists(out_csv):
354                 logger.error(
355                     "WASB native finished but expected CSV not found at %s", out_csv
356                 )
357                 return None
358             logger.info("WASB trajectory CSV: %s", out_csv)
359
360             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
361             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
362             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
363             if not self.cfg.keep_frames and frames_dir:
364                 logger.info(
365                     "Cache hygiene: removing frame cache for %s "
366                     "(set indexing.wasb.keep_frames=true to retain).",
367                     key,
368                 )
369                 self._rm_rf(frames_dir)
```

8. user (2026-07-07T04:11:25.503Z)

```
1049:def run_video_nvdec_resident(
1050-    video_path: str,
1051-    weights: str,
1052-    out_csv: str,
1053-    sport: str = "badminton",
1054-    limit: int = 0,
1055-    batch_size: int = 8,
1056-    log_every_batches: int = 50,
```

```
1057-    cache_dir: "str | None" = None,
1058-    flush_every: int = 1000,
1059-    fresh: bool = False,
1060-) -> str:
1061-    """GPU-resident path (#506): NVDEC-decode ``video_path`` on the GPU and feed
1062-    warped+normalized frames straight to the detector ? decode, resize, and
1063-    normalize never touch the host. CUDA-only by construction.
1064-
1065-    NOT byte-identical to the ``cpu`` decode paths (NVDEC's YUV?RGB and the GPU
1066-    warp differ from cv2 at float precision) ? equivalence is trajectory-level
1067-    (the #506 unit gate: 96.3% visibility agreement, median 0.036 px) and
1068-    rally-F1-level (the golden gate), so the detection cache is keyed by
1069-    ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
1070-    cache + tracker tail as ``run()`` otherwise.
1071-    """
1072-    import time
1073-
1074-    import numpy as np
1075-    import torch
1076-
1077-    from detectors import build_detector
1078-
1079-    _ensure_np_inf()
1080-    if not torch.cuda.is_available():
1081-        raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
1082-
1083-    cfg = _build_cfg(weights, sport, "cuda")
1084-    frames_in = int(cfg["model"]["frames_in"])
1085-    input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
1086-    output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
1087-    out_scales = list(cfg["model"]["out_scales"])
1088-
1089-    fe = NvdecResidentFrontend(video_path, input_wh)
1090-    try:
1091-        total = min(fe.count, limit) if limit else fe.count
1092-        if total < frames_in:
1093-            raise SystemExit(f"need >= {frames_in} frames, video has {total}")
1094-        windows_total = total - frames_in + 1
1095-        logger.info(
1096-            f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
1097-            f"{windows_total} windows"
1098-        )
1099-
1100-        # ---- cache setup / resume decision (mirrors run(); keyed nvdec_resident) --- #
1101-        if cache_dir is None:
1102-            cache_dir = default_cache_dir(out_csv)
1103-        os.makedirs(cache_dir, exist_ok=True)
1104-        det_jsonl, _ = _cache_paths(cache_dir)
1105-        cache_key = "video:" + osp.abspath(video_path)
1106-        manifest = None if fresh else read_manifest(cache_dir)
1107-        resume = manifest_compatible(
1108-            manifest or {},
1109-            frames_dir=cache_key,
```

9. user (2026-07-07T04:11:26.528Z)

No matches found

10. user (2026-07-07T04:12:16.130Z)

Structured output provided successfully